

NativePlanning 设计方案

NativePlanning 将用户的一句模糊本地生活需求理解为"可执行目标"，而不是简单关键词搜索。

1. 项目背景

本地生活场景中，用户常以一句话表达需求，例如"今天晚上带孩子出去玩""待会和老婆吃烛光晚餐"。传统搜索要求用户自行筛选场馆、核对营业时间、比较餐厅、确认座位，最终还要手动拼接行程。

NativePlanning 尝试构建一个本地生活活动规划 Agent：从自然语言理解出发，完成候选召回、可行性过滤、多维排序、计划生成，到确认预订和分享消息，提供端到端的闭环体验。

2. 目标用户与用户痛点

目标用户： 家庭亲子 / 情侣约会 / 朋友聚会 / 临时想安排半天活动的用户

用户痛点：

痛点	描述
不知道去哪	选择困难，需要从大量商户中筛选
多维信息分散	需同时考虑地点、营业时间、距离、价格、适合人群
手动拼接行程	活动 + 餐饮 + 出行时间需要人工计算
临时修改成本高	"太远了""换个餐厅"需要重头搜索
方案缺可执行性	没有订票、预约、分享等操作

3. 规划策略

3.1 意图解析（Intent Parsing）

层次	触发条件	职责
LLM Intent Parser	配置了 OPENAI_API_KEY	调用 OpenAI-compatible API（DeepSeek 等），输出结构化 UserIntent（group_type、duration、meal_policy、location_hint、time_period 等）
Rules Fallback	无 API key 或 LLM 失败	基于关键词/正则匹配，产出相同 UserIntent 结构；所有 Demo 场景均可完整运行
Post-process	统一执行	时间规范化（datetime_parser）、餐饮策略修正（meal_policy）、距离约束提取、显式地点识别、revision_scope 清除

3.2 计划生成模式（Plan Generation）

模式	触发条件	链路
activity_with_meal	默认（family / couple / friends）	场馆召回 → 餐厅召回 → 营业时间 hard gate → 多维排序 → 时间线生成 → top-N 方案
meal_only	用户明确只吃饭（如"烛光晚餐""吃火锅"）	餐厅召回 → 餐饮偏好过滤 → 排队/座位检查 → 排序 → 预约
no_meal / activity_only	meal_policy = excluded（"不吃饭"）	场馆召回 → 营业时间过滤 → 票种/距离/人群适配 → 排序 → 订票；餐饮链路完全关闭
revision	用户补充指令（"太远了""换个餐厅"）	保留原计划上下文；按 revision_scope（restaurant_only / venue_only / distance_update）局部替换，不重建整个方案

3.3 排序（Ranking）

5 维度综合评分：

维度	权重方向	说明
适配度	高	人群类型（family/couple/friends）与场馆标签匹配

维度	权重方向	说明
距离	高	距离越近得分越高；营业时间冲突有 -0.35 惩罚
氛围	中	romantic / lively / quiet 与 group_type 匹配
价格合理性	中	结合 group_type 的价位期望
新颖度	低	避免重复推荐同一商户

4. 工具调用链（Tool Call Chain）

User

→ Intent Parser（LLM / Rules）

→ DateTime Normalization（今天/明天/早上/待会 → 具体日期时间）

→ Venue Search（类型/距离/人群过滤）

→ Restaurant Search（cuisine/人群/价位过滤）

→ Opening Hours Gate（营业时间 hard gate，不可用则 feasible=False）

→ Constraint Solver（开园等待调整 / 时间冲突修复）

→ Ranker（5 维评分，top-N 候选）

→ Itinerary Builder（活动+餐饮+出行缓冲组合为时间轴）

→ Executor（book_venue / reserve_restaurant / create_order）

→ Message Agent（生成中文分享文案）

每一步均通过 ToolTrace 记录（工具名、参数、响应、耗时），在 UI 中可折叠展示，支持评委审阅完整决策链。

5. 异常处理（Exception Handling）

异常场景	处理机制
闭馆 / 营业时间冲突	Opening-hours gate 标记 feasible=False ；在同类型中搜索可行替代；无替代则输出 warning
开始时间早于开门时间	Constraint Solver 检测 start_time < open_time ；若等待时长合理（≤2h），自动调整入场时间并输出说明；否则推荐全天开放场馆
餐厅无座 / 排队过长	先在同 cuisine 中找可用替代；失败则跨 cuisine fallback，并在方案中说明替代原因

异常场景	处理机制
用户说"不吃饭"	解析为 meal_policy=excluded ，完全关闭餐饮召回链路； 分享消息剔除餐饮相关词汇
用户说"换个餐厅"	解析为 revision_scope=restaurant_only ，仅替换餐厅段， 场馆和时间轴保持不变
用户说"太远了"	更新距离约束，重新召回；输出 warning 说明原目标距离较远， 已推荐更近替代
无 API key	自动降级为 rule/mock 模式，UI 显示 [rule-based] 徽章； 全部功能正常运行
无完全匹配方案	展示最近可行方案（semantic fallback：按类型扩大召回）， 并在推荐理由中说明匹配限制

6. 功能模块（Feature Modules）

模块	职责
Planning Agent	意图解析 → 候选召回 → 可行性过滤 → 多维排序 → 时间线生成；支持 4 种计划模式
Revision Agent	接收用户补充指令，解析 revision_scope，局部替换餐厅/场馆/距离约束， 保留原计划上下文
Execution Agent	执行预订操作（订票/预约/下单），处理失败并 fallback，返回 booking ID 和下一步提醒
Explainability Layer	ToolTrace 记录完整工具调用链；方案卡片展示推荐理由、评分维度、 feasibility 状态和 warning
Mock Local-Life Tool Layer	模拟场馆搜索、餐厅搜索、营业时间查询、票种/座位/排队、预订/预约接口； 接口契约与真实 API 对齐， 可替换为真实商户/地图/排队/券包/预订 API
Streamlit Demo UI	意图解析面板（[LLM]/[rule-based] 徽章）、多方案选择器、评分卡片、 时间轴、工具追踪、两步确认执行、分享文案生成（main 分支）
TypeScript / Next.js 14 前端	App Router SPA；状态机（idle → generating → plan_ready → revising → executing → done）；8 个组件（IntentPanel / PlanCard / PlanSelector / Timeline / ToolTrace / ExecutionResult / ShareMessage / RevisionInput）； 部署于 Vercel（ https://native-planning.vercel.app ）

7. 技术架构

Frontend: Next.js 14 + TypeScript (主前端, Vercel 部署)
 Streamlit (备用演示 UI, main 分支)
Backend: FastAPI + Pydantic v2 schemas (HuggingFace Spaces 部署)
Workflow: Python planning pipeline (src/workflow/)
Tool Layer: In-memory MockAPI (src/mock_api/) – 可替换真实 API
LLM: OpenAI-compatible (DeepSeek 等国内 API)
Testing: pytest, 321 tests, 无外部 API 调用

部署地址:

- 前端 (Next.js): <https://native-planning.vercel.app>
- 前端 (Streamlit): <https://nativeplanning.streamlit.app>
- 后端: <https://aisakamai-nativeplanning.hf.space>

双后端模式:

- In-process 模式 (Streamlit 默认): 直接调用 Python 规划链
- HTTP 模式: FastAPI 独立部署, Next.js 前端通过 /api/:path* 代理请求

8. 数据层

当前使用纯内存 Mock 数据, 无外部数据库依赖:

- **场馆** (18 个): 类型、营业时间、票种、人群适配、距离、排队时长、评价标签
- **餐厅** (16 个): cuisine 标签、价位、座位状态、排队时长、营业时间、氛围标签
- **优惠券 / 套餐**: 绑定场馆或餐厅的折扣信息
- **评价标签**: specialty_tags (网红/情侣/亲子)、recommended_dishes

所有字段定义与真实本地生活平台 API 对齐, mock layer 可无缝替换为真实数据源。

9. 美团技术思路对齐

美团方向	NativePlanning 实现
GENE 场景要素拆解	UserIntent 拆解为 group_type / time_period / meal_policy / location_hint / duration 等要素, 覆盖场景 (G)、环境 (E)、需求 (N)、情感 (E) 维度

美团方向	NativePlanning 实现
搜推链路：召回→过滤→排序→解释	Venue/Restaurant Search → Opening Hours Gate → 5-dim Ranker → reasons/warnings 输出
可解释推荐	方案卡片展示各维度评分和推荐理由，而非黑盒排序
Badcase@3 评测	8 条验收用例覆盖边界场景（闭馆/开园等待/no-meal/revision），可逐条验证
Agent 工具调用	ToolTrace 记录完整调用链，与 VitaBench-like 本地生活 Agent 评测框架对齐

10. 评测方法

指标	定义
Intent Accuracy	解析结果与用户意图的字段匹配率（group_type / meal_policy / duration）
Top-1 Relevance	top1 方案与用户场景的适配评分
Feasibility	top1 方案 feasible=True 且无营业冲突
Constraint Satisfaction	硬约束（距离/开园时间/不吃饭）均被满足
Revision Correctness	revision 后只改动 scope 内部分，其余不变
Execution Completion	执行后有 booking ID + 分享消息
Badcase@3	闭馆/早于开门/餐厅无座 3 种异常均能自动修复

8 条 Demo 验收用例：

#	输入	验证重点
1	今天晚上带孩子去亲子乐园	family，夜间可行场馆，top1 feasible=True
2	待会和老婆去吃烛光晚餐	meal_only，romantic 餐厅，无 activity 段
3	明天去动物园，不吃饭	meal_policy=excluded，no 餐厅，share 消息无餐饮词
4	今晚和朋友吃火锅	friends，hotpot 餐厅，reservation
5	明天早上7点带孩子去动物园	activity start ≥ open_time，warning 说明等待
6	明天和老婆去看展，然后吃日料	exhibition venue，japanese 餐厅

#	输入	验证重点
7	用例 6 方案 → 换个餐厅	venue 不变，只换餐厅
8	明天带孩子去动物园 → 太远了	距离缩小，warning 说明替代原因

11. 已知限制

- 当前使用 mock 数据，未连接真实美团/高德 API；距离、排队、座位均为模拟值
- 无真实支付；预订结果为 mock booking ID
- 单 session，无用户登录和历史偏好记忆
- 地图距离为 mock 固定值，非实时计算

12. 后续迭代

- 接入真实地图/商户/排队/券包/预订 API（mock tool layer 已预留接口契约）
- 用户历史偏好与长期记忆
- 多城市扩展
- 在线 badcase 收敛与评测